

ENGINEERING PHYSICS CALCULATOR

Presented by :

K.John Wilson – 25A31A04N0

G.Pradeep – 25A31A04M9

I.Sailaja – 25A31A04K2

K.Nitya – 25A31A04K3



CASIO SCIENTIFIC CALCULATOR

- ❖ A mini-project developed using C language.
- ❖ Implements Scientific , matrix , and equation-solving features.
- ❖ Also implements arithmetic operations.

PROJECT OBJECTIVE :

- ❑ Create a menu-driven calculator.
- ❑ Implement reusable functions.
- ❑ Support advanced mathematics.
- ❑ Simulate CASIO calculator behavior.



KEY FEATURES :

- ▶ Arithmetic operations
(any count)
- ▶ Scientific functions
ex: calculates values of $\sin 30$, $\cos 45$, etc..
- ▶ Matrix calculations
ex: mat addition, subtraction, multiplication, inverse
- ▶ Equations solvers
ex: solves ax^2+bx+c
- ▶ Ans memory feature
ex: stores last result



Program structure

- Header files and libraries
- Global “Ans” variable
- Modular functions
- Infinite menu loop
- Switch-case logic



“Ans” MEMORY

FEATURES:

- ▶ Global variable → double Ans
- ▶ Stores last calculated results
- ▶ Accessible in all operations
- ▶ Mimics CASIO calculator memory



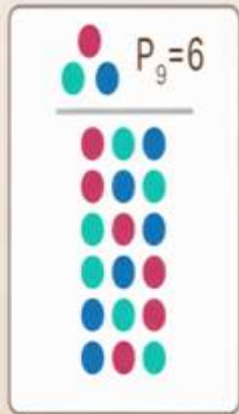
Basic Arithmetic Operations :

- ▶ Addition
- ▶ Subtraction
- ▶ Multiplication
- ▶ Division
- ▶ Supports multiple inputs using loops
(included the special case)



COMBINATIONS AND STATISTICS

Permutation & Combination



Permutation



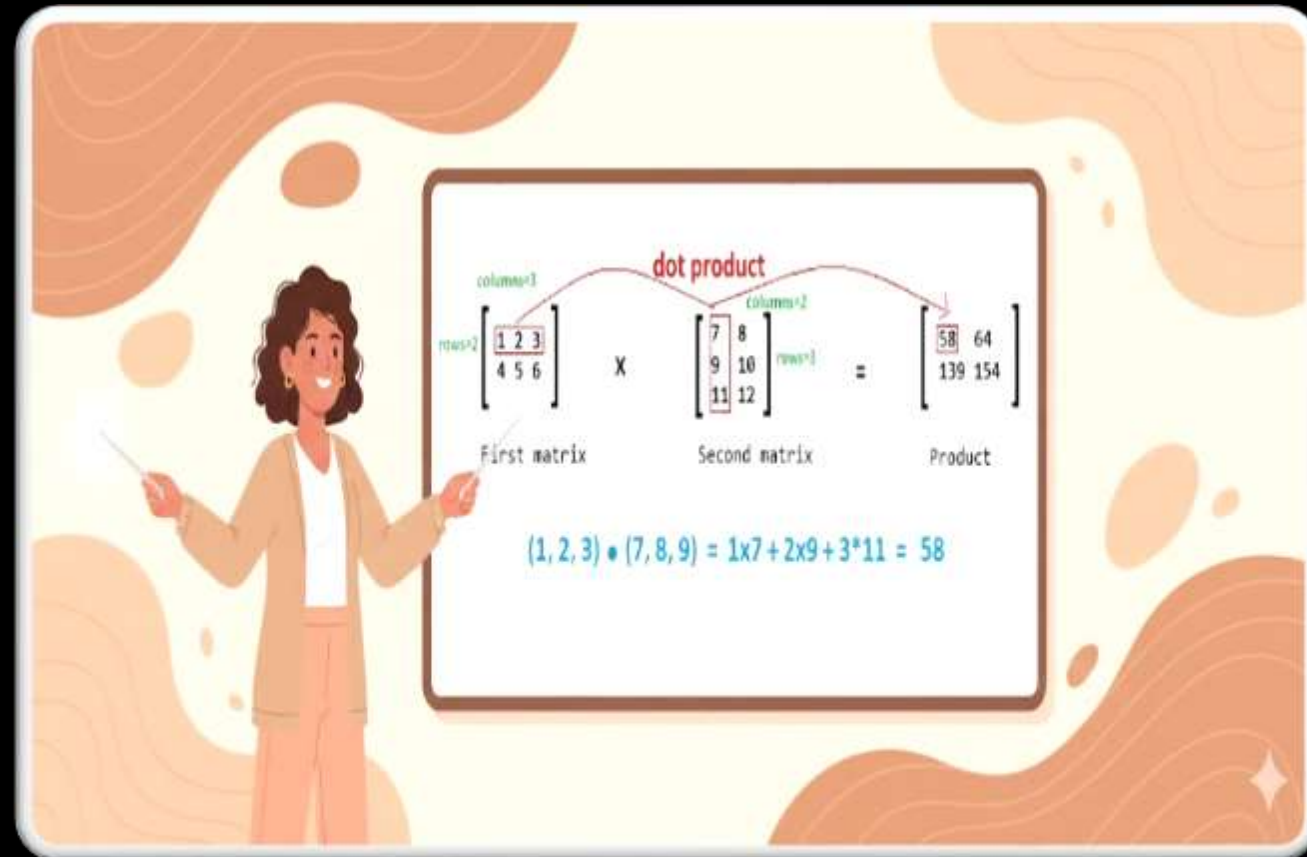
Combination



- ▶ Factorial of a number
- ▶ nPr (permutations) and nCr (combinations)
- ▶ Mean Calculation
- ▶ Max and Min operations
(finding maximum and minimum among the given numbers)

MATRIX OPERATIONS

- ▶ MATRIX ADDITION
(Must enter size of the matrices)
 - ▶ MATRIX MULTIPLICATION
(Based on 2 cases)
 - ▶ INVERSE OF THE MATRIX
 - ▶ DIMENSION VALIDATION
- The code is declared globally
i.e., Outside of the main function
 - Call the respective functions when task is initialized



EQUATION SOLVERS:

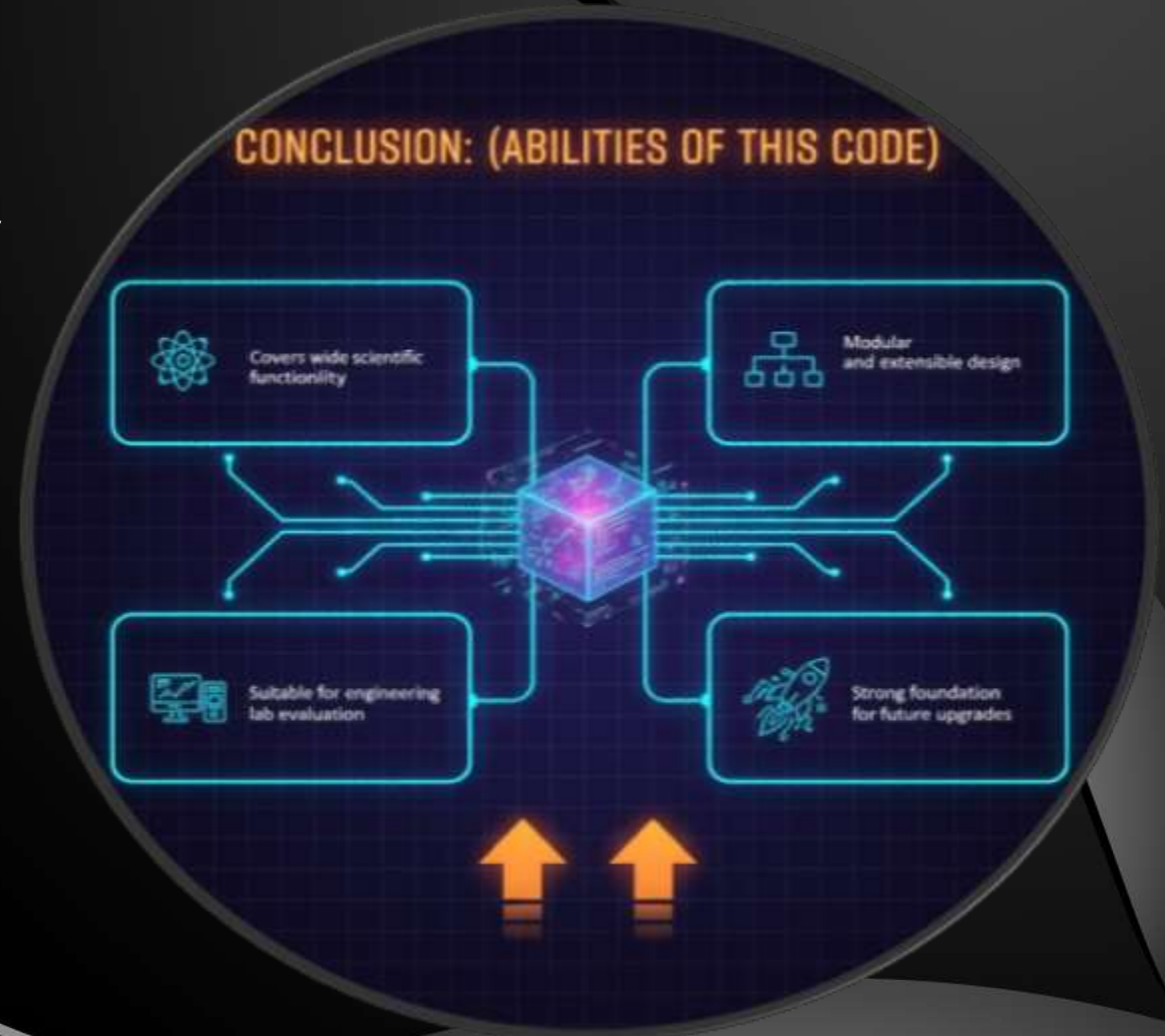
- ▶ Solves Quadratic equations using discriminants
(gives roots of the Q.E using 3 conditions)
- ▶ Cubic equations using Newton-Raphson method
- ▶ Handles real and complex roots
 - The code is declared globally
i.e., Outside of the main function
 - Code is called by function present in the switch case



CONCLUSION:

(Abilities of this code)

- ▶ Covers wide scientific functionality
- ▶ Modular and extensible design
- ▶ Suitable for engineering lab evaluation
- ▶ Strong foundation for future upgrades



Code

```
#include <stdio.h>
#include <math.h>

double Ans = 0; //Stores last answer

// =====
MATRIX INVERSE (2x2 ONLY)
=====
void inverse2x2() {
    double a, b, c, d;
    printf("Enter elements of 2x2
matrix:\n");
    scanf("%lf %lf", Ca, Cb);
    scanf("%lf %lf", Cc, Cd);

    double det = a*d - b*c;

    if(det == 0) {
        printf("Matrix is singular! Inverse not
possible.\n");
        return;
    }

    printf("Inverse matrix:\n");
    printf("%.4lf %.4lf\n", d/det, -b/det);
    printf("%.4lf %.4lf\n", -c/det, a/det);
}

// ===== QUADRATIC
ROOTS =====
```

```
void quadratic() {
    double a,b,c;
    printf("Enter a, b, c: ");
    scanf("%lf %lf %lf", Ca, Cb, Cc);

    double D = b*b - 4*a*c;

    if(D > 0) {
        double r1 = (-b + sqrt(D)) / (2*a);
        double r2 = (-b - sqrt(D)) / (2*a);
        printf("Real and distinct
roots:\n%.4lf, %.4lf\n", r1, r2);
        Ans = r1;
    }
    else if(D == 0) {
        double r = -b / (2*a);
        printf("Real and equal roots:
%.4lf\n", r);
        Ans = r;
    }
    else {
        double real = -b / (2*a);
        double imag = sqrt(-D) / (2*a);
        printf("Complex roots:\n%.4lf +
%.4lfi\n%.4lf - %.4lfi\n", real, imag, real,
imag);
        Ans = real;
    }
}
```

```
// ===== CUBIC
ROOTS (NUMERICAL NEWTON
METHOD) =====
double cubic_f(double a,double
b,double c,double d,double x){
    return a*x*x*x + b*x*x + c*x + d;
}

double cubic_df(double a,double
b,double c,double x){
    return 3*a*x*x + 2*b*x + c;
}

void cubic() {
    double a,b,c,d;
    printf("Enter coefficients a b c d: ");
    scanf("%lf %lf %lf %lf",Ca,Cb,Cc,Cd);

    double x = 1;
    for(int i=0;i<10000;i++){
        double fx = cubic_f(a,b,c,d,x);
        double fdx = cubic_df(a,b,c,x);
        x = x - fx/fdx;
        Ans = x;
    }

    printf("One real root = %.6lf\n", x);
    printf("All the other remaining roots
are complex roots....\n");
}
```

```
// ===== MATRIX ADD
=====
void matrix_add() {
    int r,c;
    printf("Enter rows and cols: ");
    scanf("%d %d",Cr,Cc);

    double A[10][10], B[10][10];

    printf("Enter Matrix A:\n");
    for(int i=0;i<r;i++)
        for(int j=0;j<c;j++)
            scanf("%lf",CA[i][j]);

    printf("Enter Matrix B:\n");
    for(int i=0;i<r;i++)
        for(int j=0;j<c;j++)
            scanf("%lf",CB[i][j]);

    printf("Sum:\n");
    for(int i=0;i<r;i++){
        for(int j=0;j<c;j++){
            printf("%.2lf", A[i][j] + B[i][j]);
        }
        printf("\n");
    }
}
```

```
// ===== MATRIX
MULTIPLY =====
void matrix_multiply() {
    int r1,c1,r2,c2;
    printf("Enter rows C cols of A: ");
    scanf("%d %d",Cr1,Cc1);
    printf("Enter rows C cols of B: ");
    scanf("%d %d",Cr2,Cc2);

    if(c1 != r2){
        printf("Invalid! Columns of A must
equal rows of B.\n");
        return;
    }

    double A[10][10], B[10][10],
    C[10][10];
    printf("Enter A:\n");
    for(int i=0;i<r1;i++)
        for(int j=0;j<c1;j++)
            scanf("%lf",CA[i][j]);

    printf("Enter B:\n");
    for(int i=0;i<r2;i++)
        for(int j=0;j<c2;j++)
            scanf("%lf",CB[i][j]);

    for(int i=0;i<r1;i++){
        for(int j=0;j<c2;j++){
            C[i][j] = 0;
```

```
        for(int k=0;k<c1;k++)
            C[i][j] += A[i][k] * B[k][j];
        }
    }

    printf("Product:\n");
    for(int
    i=0;i<r1;i++){
        for(int j=0;j<c2;j++)
            printf("%.2lf ",C[i][j]);
        printf("\n");
    } }

    long long factorial(int n) {
        long long fact = 1;
        for(int i = 1; i <= n; i++)
            fact *= i;
        return fact;
    }

    int main() {
        int choice;

        while (1) {
            printf("\n===== CASIO
SCIENTIFIC CALCULATOR
=====\\n");
            printf("Last Ans = %.4lf\\n", Ans);
            printf("1. Addition (any number)\\n");
            printf("2. Subtraction\\n");
```

```

printf("3. Multiplication\n");
printf("4. Division\n");
printf("5. Power chain\n");
printf("6. Square root\n");
printf("7. Factorial\n");
printf("8. log10\n");
printf("9. ln\n");
printf("10. Sin\n");
printf("11. Cos\n");
printf("12. Tan\n");
printf("13. nPr\n");
printf("14. nCr\n");
printf("15. Mean\n");
printf("16. Max\n");
printf("17. Min\n");
printf("18. Matrix Addition\n");
printf("19. Matrix Multiplication\n");
printf("20. Matrix Inverse 2x2\n");
printf("21. Quadratic Roots\n");
printf("22. Cubic Equation (1 real
root)\n");
printf("23. Use Ans\n");
printf("0. Exit\n");
printf("Enter choice: ");
scanf("%d", Cchoice);

if (choice == 0) break;

int n;
double num, result;

```

```

switch (choice){

    case 23:
        printf("Ans = %.4lf\n", Ans);
        continue;

    case 1:
        printf("How many numbers? ");
        scanf("%d", Cn);
        result = 0;
        for(int i=1;i<=n;i++){
            scanf("%lf",Cnum);
            result += num;
        }
        Ans = result;
        printf("Sum = %.4lf\n", result);
        break;

    case 2:
        printf("How many numbers? ");
        scanf("%d", Cn);
        scanf("%lf", Cresult);
        for(int i=2;i<=n;i++){
            scanf("%lf",Cnum);
            result -= num;
        }
        Ans = result;
        printf("Difference = %.4lf\n",
result);

```

```

        break;

    case 3:
        printf("How many numbers? ");
        scanf("%d", Cn);
        result = 1;
        for(int i=1;i<=n;i++){
            scanf("%lf",Cnum);
            result *= num;
        }
        Ans = result;
        printf("Product = %.4lf\n",
result);
        break;

    case 4:
        printf("How many numbers? ");
        scanf("%d", Cn);
        scanf("%lf",Cresult);
        for(int i=2;i<=n;i++){
            scanf("%lf",Cnum);
            if(num==0){
                printf("Error! Division by
zero.\n");
                break;
            }
            result /= num;
        }
        Ans = result;

```



```

        printf("Quotient = %.4lf\n",
result);
        break;

case 5: {
    int count;
    printf("How many numbers? ");
    scanf("%d", Ccount);

    double arr[20];
    for(int i=0;i<count;i++)
        scanf("%lf",Carr[i]);

    double p = arr[0];
    for(int i=1;i<count;i++)
        p = pow(p, arr[i]);

    Ans = p;
    printf("Result = %.4lf\n", p);
    break;
}

case 6:
    printf("Enter number: ");
    scanf("%lf", Cnum);
    Ans = sqrt(num);
    printf("sqrt = %.4lf\n", Ans);
    break;

case 7:

```

```

    printf("Enter number: ");
    scanf("%lf", Cnum);
    Ans = factorial(num);
    printf("Factorial = %.0lf\n", Ans);
    break;

case 8:
    printf("Enter number: ");
    scanf("%lf", Cnum);
    Ans = log10(num);
    printf("log10 = %.4lf\n", Ans);
    break;

case 9:
    printf("Enter number: ");
    scanf("%lf", Cnum);
    Ans = log(num);
    printf("ln = %.4lf\n", Ans);
    break;

case 10:
    printf("Enter degrees: ");
    scanf("%lf", Cnum);
    Ans = sin(num * M_PI/180);
    printf("sin = %.4lf\n", Ans);
    break;

case 11:
    printf("Enter degrees: ");
    scanf("%lf", Cnum);

```

```

    Ans = cos(num * M_PI/180);
    printf("cos = %.4lf\n", Ans);
    break;

case 12:
    printf("Enter degrees: ");
    scanf("%lf", Cnum);
    Ans = tan(num * M_PI/180);
    printf("tan = %.4lf\n", Ans);
    break;

case 13: {
    int n, r;
    printf("Enter n, r: ");
    scanf("%d %d", Cn, Cr);
    Ans = factorial(n) / factorial(n-
r); printf("nPr = %.0lf\n", Ans);
    break;
}

case 14: {
    int n, r;
    printf("Enter n, r: ");
    scanf("%d %d", Cn, Cr);
    Ans = factorial(n) /
(factorial(r)*factorial(n-r));
    printf("nCr = %.0lf\n", Ans);
    break;
}

```

```

case 15:
    printf("How many numbers? ");

    scanf("%d", &n);
    result = 0;
    for(int i=1; i<=n; i++){
        scanf("%lf", &num);
        result += num;
    }
    Ans = result/n;
    printf("Mean = %.4lf\n", Ans);
    break;

case 16:
    printf("How many numbers? ");
    scanf("%d", &n);
    scanf("%lf", &result);
    for(int i=2; i<=n; i++){
        scanf("%lf", &num);
        if(num > result) result = num;
    }
    Ans = result;
    printf("Max = %.4lf\n", Ans);
    break;

case 17:
    printf("How many numbers? ");
    scanf("%d", &n);
    scanf("%lf", &result);
    for(int i=2; i<=n; i++){
        scanf("%lf", &num);
        if(num < result) result = num;
    }

    Ans = result;
    printf("Min = %.4lf\n", Ans);
    break;

case 18:
    matrix_add();
    break;

case 19:
    matrix_multiply();
    break;

case 20:
    inverse2x2();
    break;

case 21:
    quadratic();
    break;

case 22:
    cubic();
    break;

default:
    printf("Invalid choice!\n");
}
}

return 0;
}

```



Thank You